

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ**

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»**

Факультет программной инженерии и компьютерной техники

**Дисциплина:
«Разработка мобильных приложений»**

**ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ
«Разработка мобильного приложения для системы умный дом»**

Выполнил:
Студент группы Р33101
Стрельбицкий Илья Павлович

Проверил:
к.т.н. Ключев Аркадий Андреевич

Санкт-Петербург
2024 г.

Оглавление

1. Введение2

2

1.2 Цели и задачи работы3

2. Техническое задание4

3. Архитектура системы6

4. Описание реализации8

4.1 Описание реализации мобильного приложения8

4.2 Описание реализации сервера9

5. Тестирование10

6. Заключение11

7. Список литературы12

1. Введение

1.1 Актуальность работы

Разработка мобильного приложения для управления системой "умный дом" является важной и нужной по следующим причинам:

- **Повышение комфорта и удобства:**

Пользователи смогут управлять устройствами умного дома из любого места и в любое время, что значительно упрощает контроль за домом.

- **Усиление безопасности:**

Приложение позволяет оперативно получать уведомления о проблемах (проникновение) и быстро реагировать на них.

- **Энергоэффективность:**

Управление освещением и отоплением через приложение позволяет оптимизировать их работу, снижая затраты на электроэнергию и ресурсы.

- **Растущий рынок:**

Рынок умных домов активно развивается. Вслед за ними растет спрос на мобильные приложения для управления умным домом.

- **Социальная значимость:**

Умные дома улучшают качество жизни, особенно для людей с ограниченными возможностями и пожилых, делая повседневные задачи более управляемыми.

- Конкурентное преимущество:

Для компаний наличие современного мобильного приложения является важным фактором конкурентоспособности, помогая привлекать и удерживать клиентов. Поскольку оно зачастую более удобно для потребителя чем сайт.

Таким образом, разработка мобильного приложения для системы "умный дом" отвечает современным потребностям, повышает комфорт, безопасность и энергоэффективность жилых помещений, что делает данный проект актуальным и востребованным.

1.2 Цели и задачи работы

Цели:

- Создание удобного приложения для управления системой «умный дом».
Обеспечить пользователям простой и интуитивно понятный способ контроля и управления всеми устройствами умного дома через мобильное приложение.
- Создать задел для работы разработанного приложения с полноценным сервером.
Разработанное приложение должно работать как будто уже имеет связь с полноценным сервером, а не с примитивной заглушкой.

Задачи:

- Анализ требований и определение функционала:
Изучить потребности пользователей и разработать перечень необходимых функций для мобильного приложения.
- Разработка пользовательского интерфейса:

Создать удобный, интуитивный и эстетически привлекательный интерфейс, обеспечивающий легкий доступ к основным функциям управления умным домом.

- Интеграция с серверным приложением:

Обеспечить совместимость приложения с сервером управляющим умным домом.

- Создание имитации сервера:

Для того, чтобы обеспечить задел для работы приложения с полноценным сервером понадобится имитация сервера, которая будет иметь интерфейс сервера, но при этом внутри устроена примитивно – без использования баз данных и прочих сложностей.

- Тестирование и отладка приложения:

Провести тестирование приложения с использованием разработанной имитации сервера.

2. Техническое задание

Требуется в качестве фронтенда сделать мобильное приложение для операционной системы Android на языке программирования Kotlin. Для создания пользовательского интерфейса должен использоваться Jetpack Compose.

Мобильное приложение должно иметь 4 динамические страницы. Структура каждой страницы – заголовок сверху, контент по середине и основное меню в нижней части. Основное меню состоит из 3 элементов, которые содержат ссылки на страницы: главной, сигнализации и настроек. Последняя 4 страница содержит информацию о выбранной комнате на главной странице. Главная страница должна показывать список комнат, которые подключены к системе умный дом в виде набора плиток с названием комнаты и иконкой. Информация о комнатах должна подгружаться с сервера в фоновом режиме. При нажатии на плитку комнаты должна открываться страница с подробной информацией о комнате. В верхней части должен находиться список

доступных функций в комнате, ниже должна находиться информация о выбранной функции. В каждой комнате разный список доступных функций, всего их доступно 3 – температура, влажность и освещенность. Еще ниже должен располагаться компонент для установки нового значения выбранной функции. Информация должна подгружаться в фоновом режиме. При установке нового значения должен отправляться запрос на сервер, содержащий новое значение.

Страница сигнализации должна содержать информацию о том работает ли сигнализация сейчас, переключатель состояния ее работы и то сработала ли сигнализация. Запросы на получение состояния сигнализации должны отправляться в фоновом режиме в независимости от того на какой странице находится пользователь. Если сигнализация сработала, то в этом случае должно появляться всплывающее уведомление, в независимости от того на какой странице находится пользователь.

Страница настроек должна содержать информацию об IP адресе и порте сервера. Также должна быть форма, которая позволит задавать пользователю значения IP адреса и порта. Все запросы, которые отправляет мобильное приложение, должны отправляться на заданный в настройках IP адрес и порт. В качестве бэкенда требуется написать упрощенную имитацию сервера на языке программирования Kotlin с использованием фреймворка Spring Boot. Для хранения данных не должна использоваться база данных, а вместо нее должны использоваться структуры данных языка Kotlin, которые хранятся в оперативной памяти.

Нужно определить первоначальное наполнение структур, которые будут использоваться для формирования ответа на эндпоинтах приложения. При этом изменения, которые вносятся запросами со стороны клиента, должны сохраняться в этих структурах данных.

То есть, например для эндпоинта получения данных о комнатах умного дома, можно определить массив объектов, инкапсулирующих информацию о комнатах. Для эндпоинта получения данных о состоянии комнаты хэш-

таблицу, где ключ это id комнаты, а значение объект, содержащий нужную информацию. Для эндпоинта получения данных о состоянии сигнализации можно использовать просто переменные с примитивами, хранящими необходимую информацию.

Передача данных между бекендом и фронтендом должна происходить по протоколу HTTP с использованием JSON для передаваемых данных и парадигмы REST.

3. Архитектура системы

Архитектура мобильного приложения (фронтенда) тривиальна, поэтому опишем архитектуру бекенда.

Для того чтобы иметь возможность горизонтального масштабирования и разбиения серверного приложения на микросервисы для каждой бизнес задачи выделим отдельный эндпоинт, что также позволит уменьшить объем передаваемых данных между клиентом и сервером (но увеличит число самих запросов).

Можно выделить следующие бизнес-процессы:

- Получение информации о комнатах, подключенных к системе умный дом.
- Получение информации о состоянии сигнализации.
- Изменение состояния работы сигнализации.
- Получение информации о доступных функциях комнаты и значения для выбранной функции.
- Изменение значения выбранной функции в выбранной комнате.

Для каждого из них выделим свой эндпоинт с заданием метода, который соответствует назначению бизнес-задачи.

Список эндпоинтов:

- GET: /room-data – получение информации о комнатах

Формат ответа от сервера:

```
[{"id":0,"title":"Гостиная","nameIcon":"living_room"}]
```

- GET: /signaling-data – получение информации о сигнализации

Формат ответа от сервера:

```
{"signalingWorkingState":"work","signalingState":true}
```

- POST: /signaling-data – переключение состояния работы сигнализации

Нет ни ответа от сервера ни передаваемого объекта клиентом.

- GET: /humidity-data/{id} – получение информации об влажности в комнате с заданным id.

Формат ответа от сервера:

```
{  
  "idRoom": 1,  
  "enableFunctions": ["humidity_room","lights_room","temperature_room"],  
  "humidity": 40.0  
}
```

- POST: /humidity-data/{id} – изменение значения влажности в комнате с заданным id.

Формат передаваемого объекта клиентом:

```
{"idRoom": 1, "humidity": 50.0}
```

- GET: /lights-data/{id} – получение информации об освещенности в комнате с заданным id.

Формат ответа от сервера:

```
{  
  "idRoom": 1,  
  "enableFunctions": ["humidity_room","lights_room","temperature_room"],  
  "lights": 90.0  
}
```

- POST: /lights-data/{id} – изменение значения освещенности в заданной комнате.

Формат передаваемого объекта клиентом:

```
{"idRoom": 1, "lights": 70.0}
```

- GET: /temperature-data/{id} – получение информации о температуре в комнате с заданным id.

Формат ответа от сервера:

```
{
  "idRoom": 1,
  "enableFunctions": ["humidity_room", "lights_room", "temperature_room"],
  "temperature": 30.0
}
```

- POST: /temperature-data/{id} – изменение значения температуры в комнате с заданным id.

Формат передаваемого объекта клиентом:

```
{"idRoom": 1, "temperature": 35.0}
```

4. Описание реализации

4.1 Описание реализации мобильного приложения

Ссылка на github – <https://github.com/stoneshik/rmp>

Мобильное приложение было реализовано на языке программирования Kotlin с использованием технологии Jetpack Compose для создания пользовательских интерфейсов на операционной системе андроид. В качестве библиотеки для работы с Http запросами используется Okhttp и для работы с форматом Json – Kotlinx.

В созданном приложении 3 основных пакета:

- 1) Client – реализует взаимодействие мобильного приложения по http и также загрузчик данных который в фоновом режиме определяет какие данные, получаемые от сервера необходимо обновить.
- 2) Navigation – реализует навигацию между страницами в приложении и основное меню
- 3) Screens – реализует страницы приложения: компоненты, объекты данных которые необходимы для их работы.

Переменные, которые хранят состояния (MutableState), инициализируются в компоненте MainScreen, после чего компонент передает их компоненту Navigation, который отвечает за навигацию в приложении. Компонент Navigation на основании того какая страница приложения открыта выбирает компонент страницы, который нужно вызвать и передает ему необходимые переменные состояний.

4.2 Описание реализации сервера

Ссылка на github – <https://github.com/stoneshik/rmp-server>

Сервер был реализован на языке программирования Kotlin с использованием фреймворка Spring Boot.

Сервер реализует обработку эндпоинтов, которые были перечислены в пункте 3 (Архитектура системы). То есть тем самым реализуется интерфейс полноценного сервера, благодаря чему достигается, то, что клиенту нет разницы используется ли полноценный сервер или имитатор.

Внутреннее устройство имитатора сервера примитивно, в нем не используется база данных для хранения данных, а используются структуры данных языка Kotlin хранимые в оперативной памяти во время работы сервера.

Для эндпоинта GET: /room-data – используется массив с объектами, которые инкапсулируют данные о комнатах.

Массив представлен на изображении ниже:

```
val dataRooms = arrayOf(  
    RoomData(1, "Гостиная", "living_room"),  
    RoomData(2, "Спальня", "bedroom"),  
    RoomData(3, "Кухня", "kitchen"),  
    RoomData(4, "Ванная", "bathroom"),  
    RoomData(5, "Студия", "studio")  
)
```

Для эндпоинтов GET: /humidity-data/{id} и POST: /humidity-data/{id} – используется хеш-таблица, в которой в качестве ключа представлен id выбранной комнаты, а в качестве значения объект, инкапсулирующий значения данных комнаты.

Хеш-таблица представлена на изображении ниже:

```
private var dataMap = mapOf(
    "1" to FunctionHumidityData(1, arrayOf("humidity_room", "lights_room", "temperature_room"), 40.0),
    "2" to FunctionHumidityData(2, arrayOf("humidity_room", "lights_room", "temperature_room"), 41.0),
    "3" to FunctionHumidityData(3, arrayOf("humidity_room", "lights_room", "temperature_room"), 42.0),
    "4" to FunctionHumidityData(4, arrayOf("lights_room", "temperature_room"), 43.0),
    "5" to FunctionHumidityData(5, arrayOf("temperature_room"), 44.0),
)
```

Для эндпоинтов GET: /lights-data/{id}, POST: /lights-data/{id} и GET: /temperature-data/{id}, POST: /temperature-data/{id} используются аналогичные хеш-таблицы.

Для эндпоинтов GET: /signaling-data и POST: /signaling-data используются переменные содержащие примитивы, хранящие необходимые данные. Со старта задано состояние «Работает» у сигнализации и «Спокойно». Спустя 30 запросов от клиента о состоянии сигнализации состояние «Спокойно» переключается в «Тревога». Затем спустя еще 10 запросов состояние «Тревога» переключается в состояние «Спокойно», после чего цикл повторяется. Так как клиент отправляет запрос раз в секунду, то приблизительное время всего цикла составляет 40 секунд.

5. Тестирование

В качестве интеграционного тестирования мобильного приложения использовалась возможность предпросмотра компонента Jetpack Composable. Пример кода продемонстрирован на изображении ниже:

```

@Preview(showBackground = true)
@Composable
fun TopBarPreview() {
    val titleTopBar = remember{
        mutableStateOf(NavigationItem.Home.title)
    }
    TopBar(titleTopBar)
}

```

Вывод кода, предоставленного выше продемонстрирован на изображении ниже:

TopBarPreview



Поскольку в разработанном приложении отсутствует сложная бизнес-логика, то отрисовка отдельных компонентов в качестве интеграционного тестирования достаточна.

Для системного тестирования был разработан на стороне бекенда имитатор полноценного сервера. Со стороны фронтенда нет разницы используется ли имитатор-заглушка или полноценный сервер, так как имитатор имеет такой же интерфейс, как и полноценный сервер.

6. Заключение

В ходе выполнения данной практической работы было успешно разработано мобильное приложения для управления умным домом и сервер имитатор для системного тестирования. Также был изучен язык программирования Kotlin и технология создания пользовательских интерфейсов для мобильных приложений с операционной системой андроид Jetpack Compose. Все поставленные цели и задачи были успешно достигнуты.

7. Список литературы

- 1) A Guide to OkHttp. (Дата посещения 23.06.2024) – <https://www.baeldung.com/guide-to-okhttp>
- 2) Building web applications with Spring Boot and Kotlin. (Дата посещения 25.06.2024) – <https://spring.io/guides/tutorials/spring-boot-kotlin>
- 3) Get started with Jetpack Compose. (Дата посещения 25.06.2024) – <https://developer.android.com/develop/ui/compose/documentation>
- 4) Get started with Kotlin (Дата посещения 23.06.2024) – <https://kotlinlang.org/docs/getting-started.html>
- 5) Введение в Jetpack Compose. (Дата посещения 23.06.2024) – <https://habr.com/ru/companies/rncb/articles/669374/>

8. Приложение

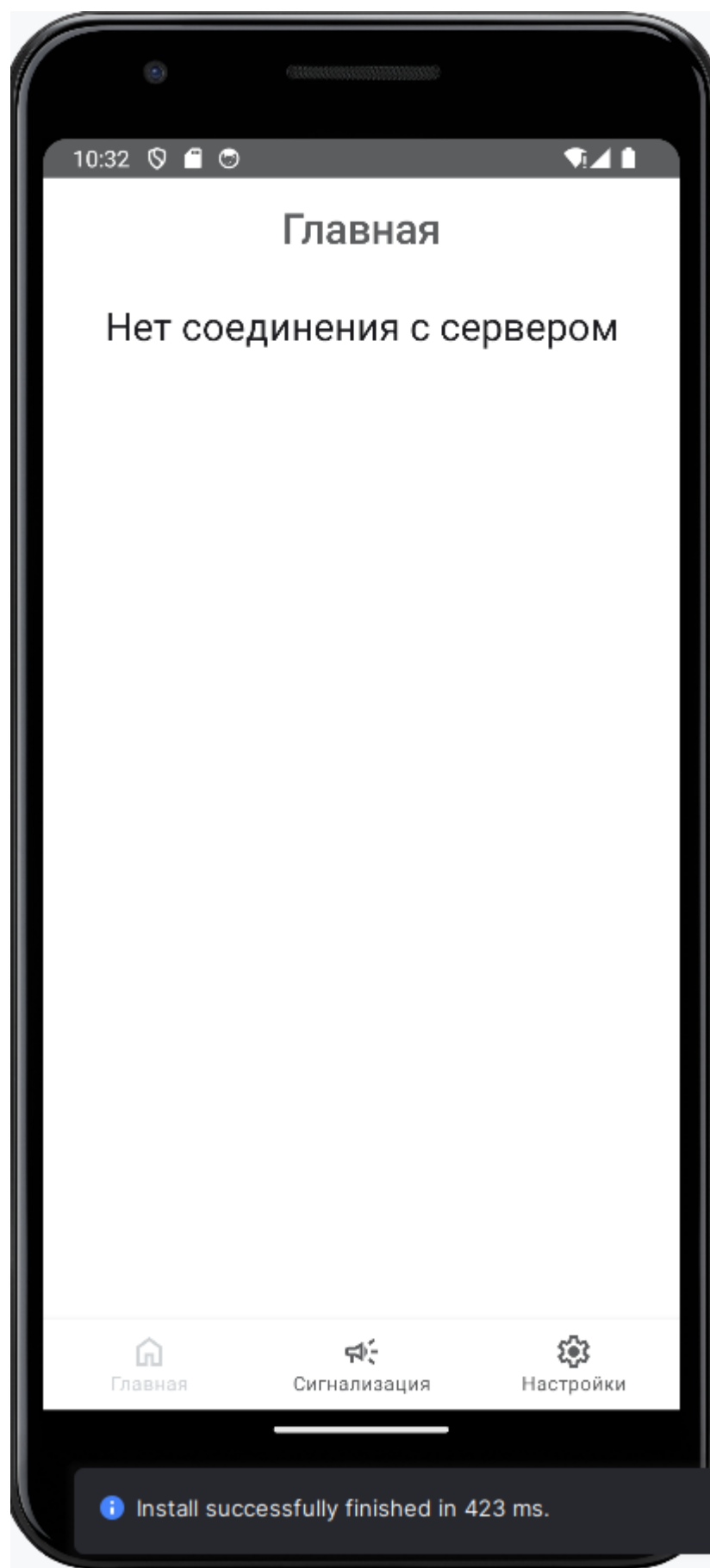


Рисунок 1. работа приложения при отсутствии соединения с сервером.

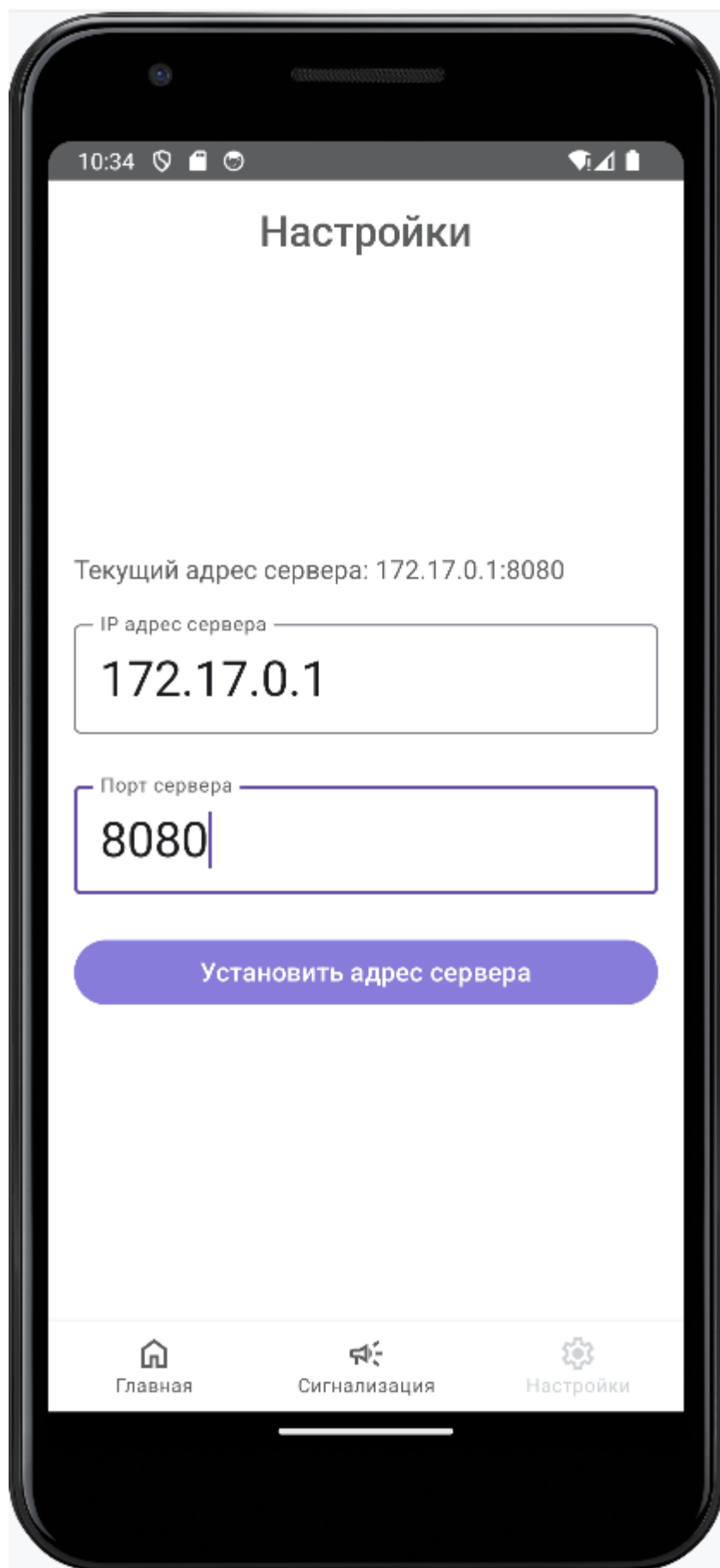


Рисунок 2. Страница настройки IP адреса сервера и порта сервера.

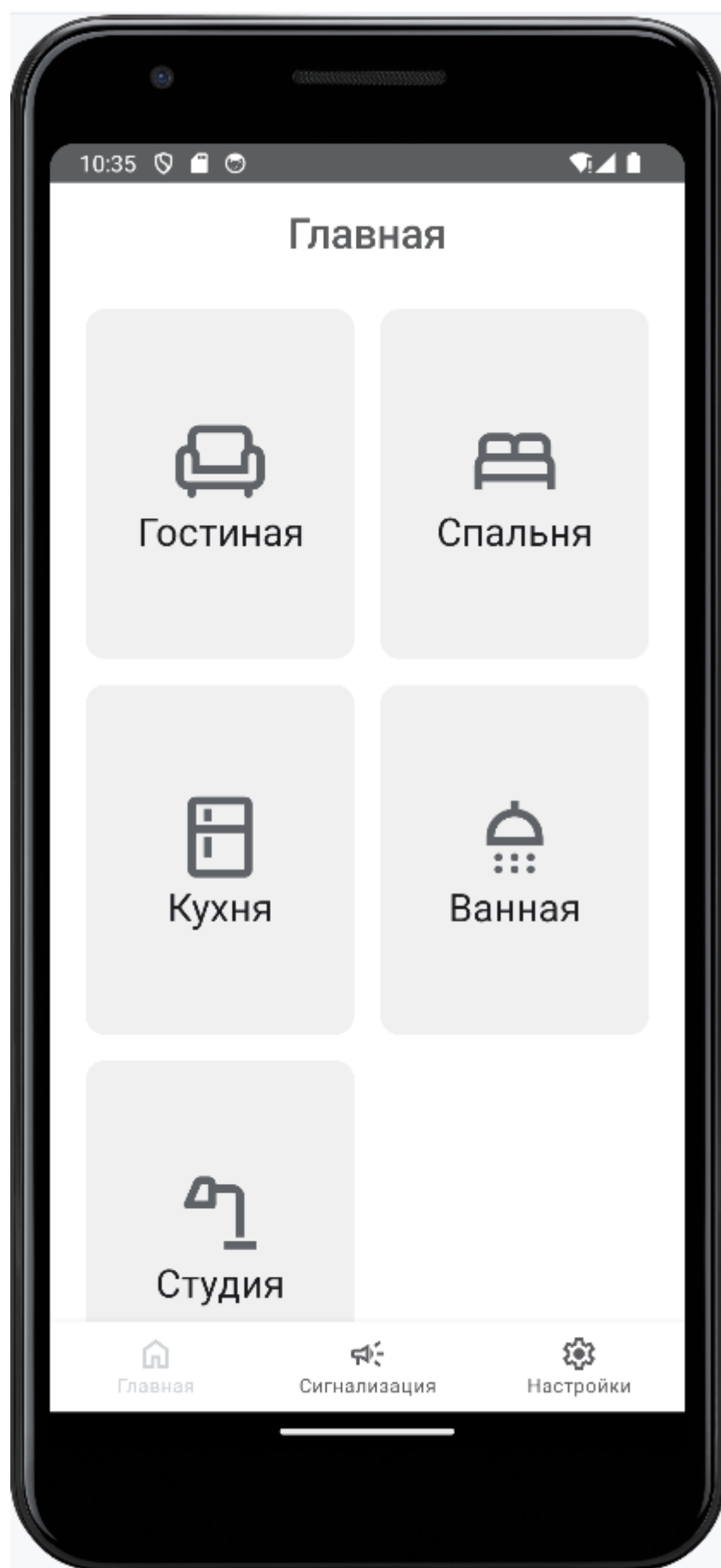


Рисунок 3. Главная страница.

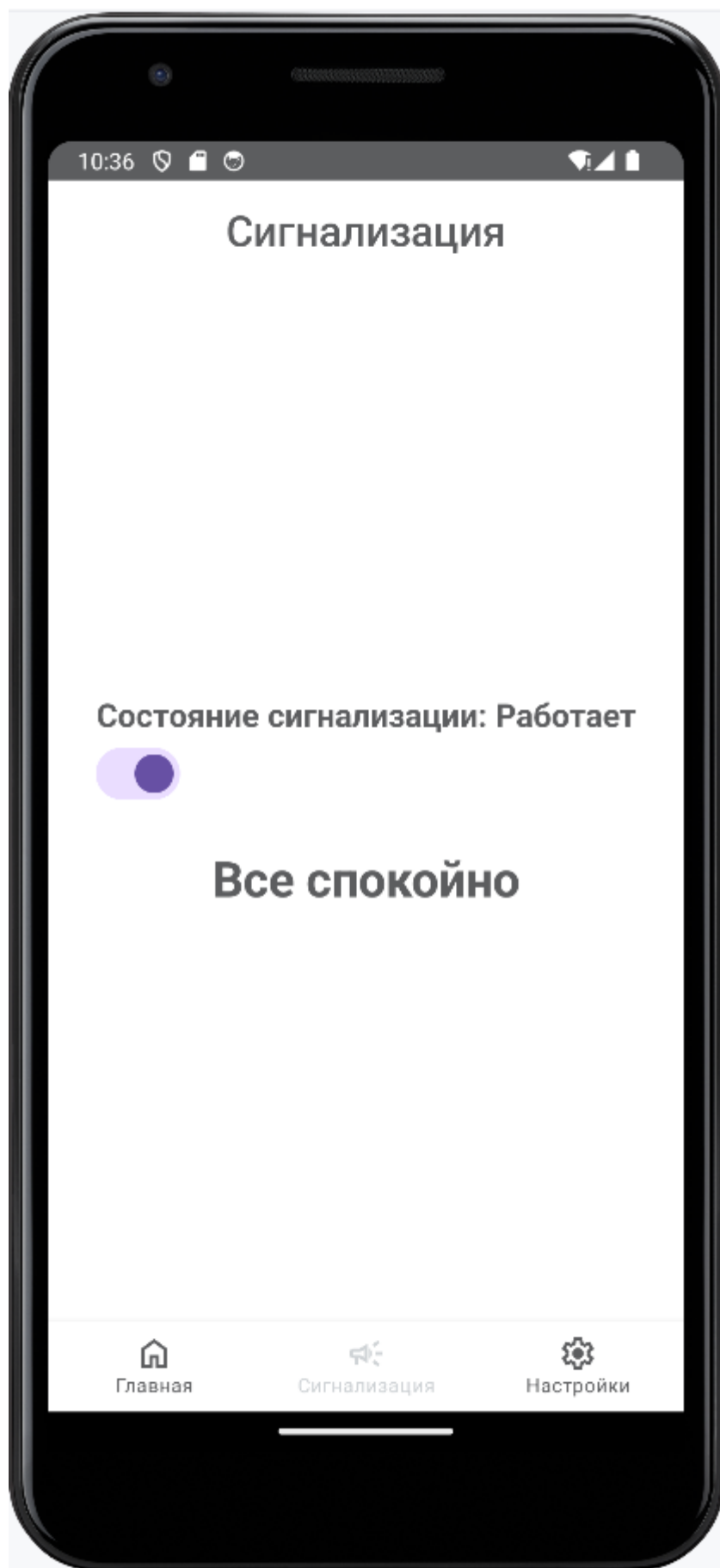


Рисунок 4. Страница сигнализации в спокойном состоянии

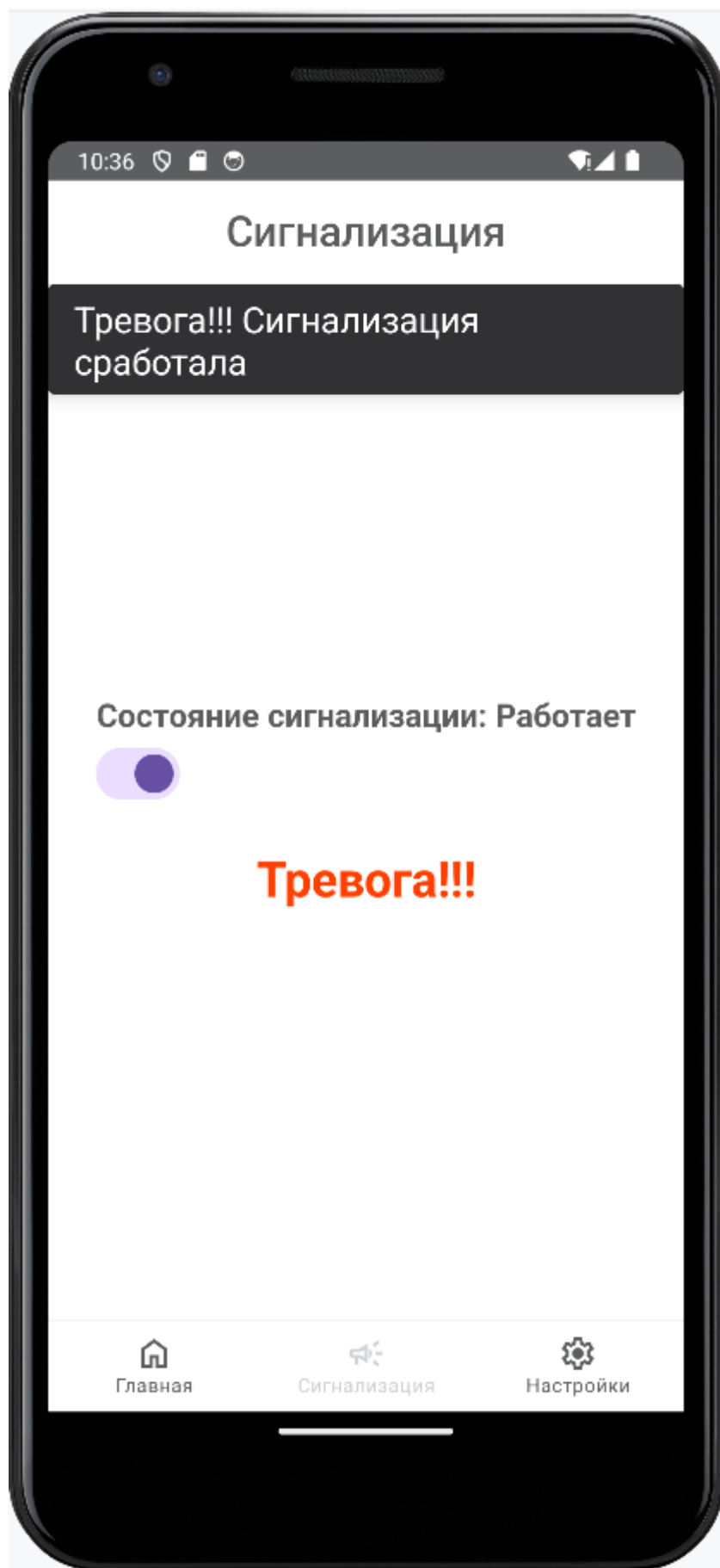


Рисунок 5. Страница сигнализации в состоянии тревоги.

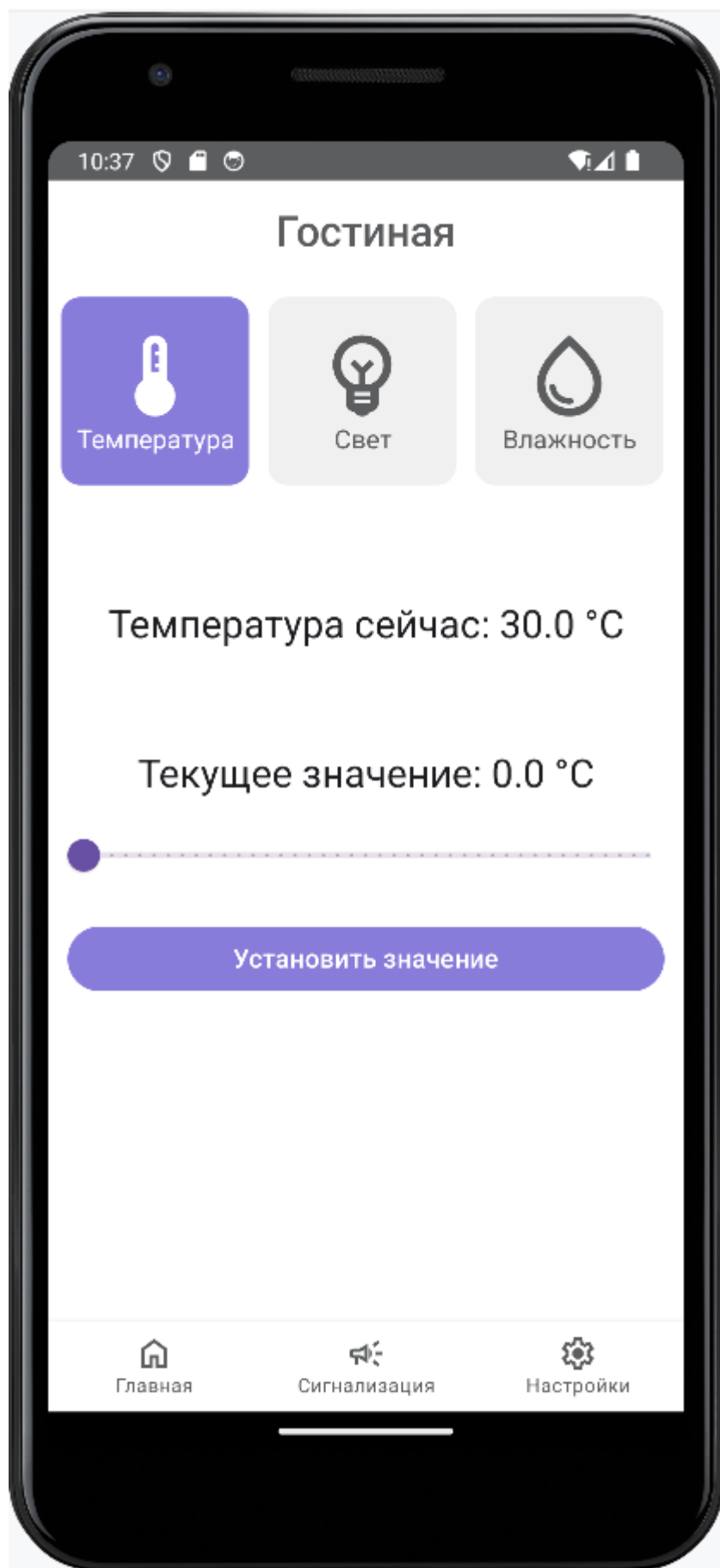


Рисунок 6. Страница выбранной комнаты (гостиной) с выбранной функцией температуры.

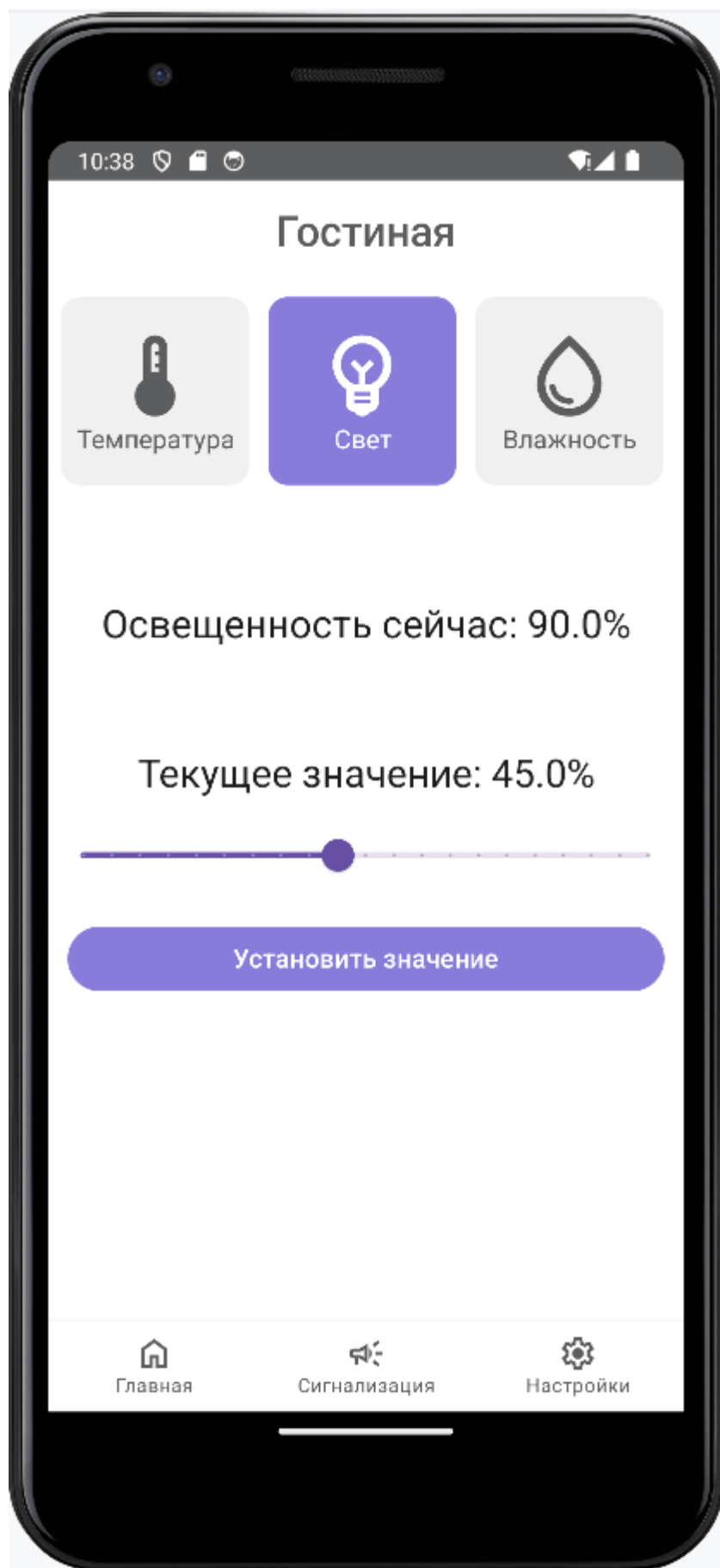


Рисунок 7. Страница выбранной комнаты (гостиной) с выбранной функцией света.

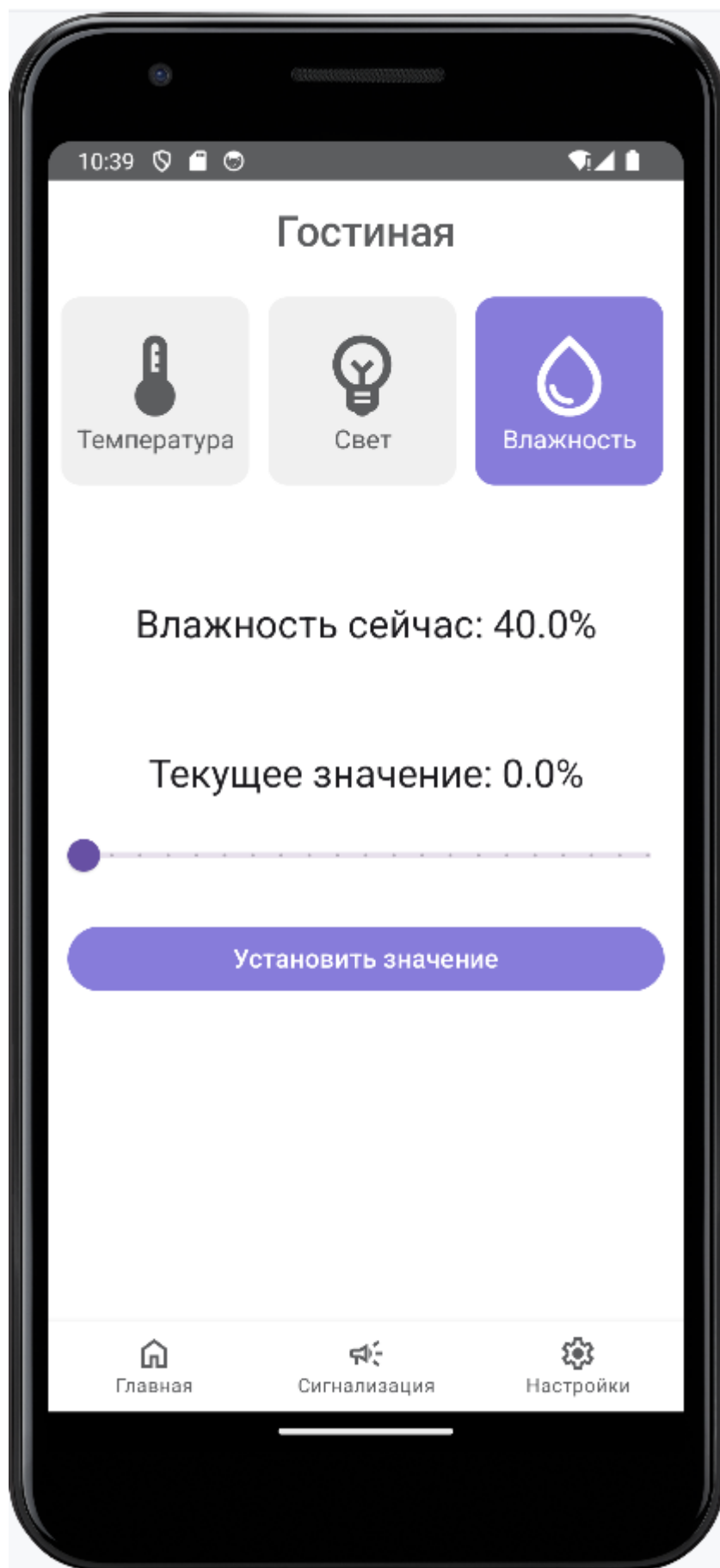


Рисунок 8. Страница выбранной комнаты (гостиной) с выбранной функцией влажности.